least it wasn't before the Arduino came along. Likewise, I was never one for a love of mathematics, which thankfully is not a prerequisite to deeply enjoy the process of writing code.

Reading This Book

Our process in each chapter will be to focus on some fundamental projects that build on the primary concepts presented in that chapter. For each project, we will begin with a project description and discuss the specific hardware needed for that project. We will also provide diagrams and illustrations for making these simple circuits and interfacing them to the Arduino board. As you read through each project, you should take notes and write in the margins—we won't be offended. Experiment, try new things, and see what happens.

The projects demonstrated in this book are meant to be prototypes, or fundamental proof-of-concept designs for a new device. We will adhere to a degree of minimalism, keeping to simple and easily obtainable hardware that supports the development of sophisticated written code. Once you have built the prototype, it can be incorporated into a final project later. We won't actually be doing that here so that we can focus on actually writing and developing code. Our examples will borrow and build on each other throughout the book, revisiting past examples when we need to as our understanding of writing code develops.

The intent with our code samples is to write compartmentalized or modular code wherever possible to allow for easy adaptability and future development. We will spend a lot of time developing our coding skills so that when it comes time to develop a new project independently, you will know where to begin. The sketches are meant to be fluid—you are encouraged to hack them—changing values, timing, pin assignments, ranges, and so on—until it no longer works. Then try to fix it. We will stick to a particular style of writing code in our samples, although we urge you to develop your own writing style that reflects the way you think and the way you want to see your code.

Wiring up the circuits for our projects is as simple and straightforward as possible, with little to no understanding of electronics necessary. As a way to reconnect our discussions of programming to the physical electronics used throughout the book, Chapter 12 will provide a brief review of some basic electronics, including how circuits work, reading schematics, and an introduction to soldering. If you find that you are struggling with hooking up the projects in the earlier chapters, you might want to jump to Chapter 12 for a refresher. Otherwise, this chapter will serve as a good summary that could help answer some questions you might not even know you had. While this might at first seem a little backwards, it has worked pretty well in my classes over the last few years.

Arduino 1.0

At the time of this writing, the Arduino developers are hard at work on a more stable, more efficient, and generally improved version of the Arduino software called Arduino 1.0. The final release version of Arduino 1.0 should be available right about the same time that this book is published. This is important because in the process of making things better, some things had to be broken. This means that some older code written under the alpha release of the Arduino software will no longer work on Arduino 1.0.

Conversely, the code in this book and images of the Arduino development environment have all been prepared using a beta release of Arduino 1.0 (`http://code.google.com/p/arduino/wiki/Arduino1`), so images of the Arduino software may appear different from the final version, some of the code in this book may not work on older versions of the software, and still other features of 1.0 were not yet fully implemented—so I couldn't write about them. There may also be other growing pains with this upgrade that we are not fully aware of at this time, so if an unusual problem crops up, then you might want to blame 1.0 and start there to figure out what's wrong.